Dipartimento di Ingegneria Elettrica Elettronica e Informatica

Corso di Laurea in Ingegneria Informatica

Anno 2025/2026

Andrea Leonardi, Simone Gucciardo, Giuseppe Aurnia

Documentazione Test

Università degli Studi di Catania — Ingegneria del Software

1. Introduzione

Il presente documento illustra la suite di test sviluppata per il sistema di gestione di un Bed & Breakfast (BnB), realizzato nell'ambito del corso di Ingegneria del Software. L'obiettivo è duplice: descrivere le scelte architetturali e progettuali adottate in fase di testing, e illustrare la copertura dei casi di test rispetto ai requisiti funzionali del sistema.

Il sistema modella l'intero ciclo di vita di una prenotazione alberghiera: dalla richiesta iniziale del cliente fino al check-out, includendo gestione della disponibilità delle camere, pagamento, modifica e cancellazione. La suite di test è stata progettata con l'obiettivo di verificare la correttezza funzionale del sistema rispetto ai requisiti specificati, garantendo la validazione delle principali operazioni del dominio applicativo e delle regole di business relative alla gestione delle prenotazioni. In particolare, i test verificano la correttezza del ciclo di vita della prenotazione, la gestione della disponibilità delle camere e la validazione degli input del sistema.

La Suite adotta un approccio ibrido, definibile Grey-Box: la progettazione dei casi di test (test design) segue logiche Black Box, in cui i casi di test sono derivati dalle specifiche funzionali del sistema senza accedere al codice sorgente; mentre l'implementazione sfrutta tecniche White Box per garantire l'isolamento ambientale tramite il reset del Singleton tra un test e l'altro. In particolare, vengono utilizzate le tecniche di partizione in classi di equivalenza, analisi dei valori limite e Test sulle transizioni di stato.

---

2. Fondamenti Teorici del Testing Adottato

2.1 Testing Black Box e Classi di Equivalenza

Il testing Black Box è una famiglia di tecniche in cui si accede al software attraverso la sua interfaccia esterna, senza alcun accesso diretto alla struttura interna del codice. Il principio fondamentale è quello della partizione in classi di equivalenza: il dominio di ingresso viene suddiviso in sottoinsiemi tali che il comportamento del sistema sia equivalente per tutti gli elementi appartenenti alla stessa partizione.

Formalmente, una classe di equivalenza è un insieme di valori di input per i quali il sistema dovrebbe produrre lo stesso comportamento atteso. Questa proprietà consente di ridurre drasticamente il numero di casi di test necessari, selezionando un solo rappresentante per ciascuna classe, nella ragionevole assunzione che se il sistema è corretto per un elemento della classe lo sarà per tutti gli altri.

Per ciascun input, si identificano tipicamente una o più classi valide (CV) e una o più classi non valide (CNV). Le classi non valide devono essere testate singolarmente, in quanto la combinazione di più input non validi potrebbe mascherare l'effettiva causa di un fallimento.

2.2 Analisi dei Valori Limite (Boundary Value Analysis)

Una specializzazione della tecnica delle classi di equivalenza è l'analisi dei valori limite. I difetti si concentrano nei pressi dei confini degli intervalli di validità. Per un intervallo chiuso [min, max], le boundary classes tipiche sono: il valore esattamente al limite inferiore (min), quello leggermente inferiore (min-1), quello esattamente al limite superiore (max) e quello leggermente superiore (max+1).

Nella suite di test del sistema BnB questa tecnica è applicata in modo sistematico, come documentato nelle sezioni seguenti. Ad esempio, per la validazione del CVV (intero a tre cifre), i valori 99 (min-1), 100 (min), 999 (max) e 1000 (max+1) costituiscono le boundary classes dell'intervallo [100, 999].

---

3. JUnit 4 e i Metodi Assert

3.1 Cos'è JUnit 4

JUnit è il framework di testing unitario più diffuso nell'ecosistema Java. Appartiene alla famiglia xUnit, un insieme di framework di testing caratterizzati dall'uso di metodi di test automatizzati e meccanismi di asserzione. JUnit 4 si basa su un insieme di annotazioni che istruiscono il motore di esecuzione dei test su come comportarsi. Le annotazioni fondamentali utilizzate nella suite del sistema BnB sono le seguenti:

```
// Indica che il metodo è un caso di test da eseguire
@Test
public void nomeDelTest() { ... }

// Variante: il test ha successo SOLO se viene lanciata questa eccezione
@Test(expected = RuntimeException.class) public
void testCasoNegativo() { ... }

// Eseguito prima di OGNI singolo test della classe (setup)
@Before
public void setUp() { ... }

// Eseguito dopo OGNI singolo test della classe (teardown / pulizia)
@After
public void tearDown() { ... }
```

Il ciclo di esecuzione di ogni singolo test in JUnit 4 è quindi sempre il seguente: viene creata una nuova istanza della classe di test, viene eseguito il metodo @Before, viene eseguito il metodo @Test, infine viene eseguito il metodo @After. La creazione di una nuova istanza per ogni test è una scelta progettuale deliberata di JUnit: garantisce che lo stato degli attributi di istanza non si propaghi da un test all'altro, favorendo l'isolamento.

Nella suite BnB, @Before viene usato per inizializzare il sistema (aggiungere camere e registrare clienti) prima di ogni test di integrazione, mentre @After viene usato per resettare il Singleton BnB tramite reflection, garantendo che ogni test parta da un sistema completamente pulito.

3.2 Il meccanismo degli Assert

Nel framework JUnit 4, i metodi assert costituiscono il meccanismo fondamentale per esprimere le aspettative di un test. Un metodo assert verifica una condizione: se la condizione è soddisfatta il test prosegue, altrimenti lancia un AssertionError che JUnit interpreta come fallimento del test. Tutti i metodi assert appartengono alla classe statica org.junit.Assert e vengono importati tramite:

```
import static org.junit.Assert.*;
```

Ogni metodo accetta opzionalmente un primo parametro di tipo String che costituisce il messaggio di errore visualizzato in caso di fallimento. L'utilizzo di messaggi descrittivi rende immediata la comprensione del motivo del fallimento senza dover analizzare il codice del test.

3.3 assertEquals — Verifica l'uguaglianza tra due valori

È il metodo assert più utilizzato nella suite. Confronta un valore atteso (expected) con un valore effettivo prodotto dal sistema (actual). Il test fallisce se i due valori non sono uguali secondo il metodo equals().

```
// Sintassi base
assertEquals(valoredAtteso, valoreEffettivo); assertEquals("Messaggio
di errore", valoreAtteso, valoreEffettivo);

// Variante per float e double: aggiunge un margine di tolleranza (delta)
assertEquals("Messaggio", valoreAtteso, valoreEffettivo, delta);
```

La variante con delta è indispensabile per i confronti tra valori in virgola mobile (float, double), poiché la rappresentazione binaria di questi tipi può introdurre piccole imprecisioni. Ad esempio, 3 * 80.0f potrebbe non produrre esattamente 240.0f ma un valore come 239.99998f. Il delta definisce la soglia massima di differenza accettabile.

Nella suite del sistema BnB, assertEquals viene utilizzato in tre contesti principali:

- Verifica degli stati della prenotazione: assertEquals("ATTESA", p.getStatoNome()) controlla che lo stato corrente corrisponda a quello atteso dopo una transizione.
- Verifica del prezzo totale: assertEquals(240.0f, p.getPrezzoTotale(), 0.001) con delta per gestire le imprecisioni float.
- Verifica degli attributi degli oggetti: assertEquals(4, c.getCapienza()) controlla che i campi siano stati inizializzati correttamente dal costruttore.

```
// Confronto di stringhe (stato prenotazione)
assertEquals("Lo stato iniziale deve essere ATTESA", "ATTESA", p.getStatoNome());

// Confronto di float con delta (prezzo soggiorno)
assertEquals("Prezzo 3 notti non corretto", 240.0f, p.getPrezzoTotale(), 0.001);
```

```
// Confronto di int (capienza camera)
assertEquals("La capienza dovrebbe essere 4", 4, c.getCapienza());

// Confronto di oggetti LocalDate (ripristino date dopo rollback)
assertEquals("Data non ripristinata", inizioOrig, p.getDataInizio());
```

3.4 assertTrue e assertFalse — Verifica condizioni booleane

assertTrue(condizione) verifica che una condizione booleana sia vera; assertFalse(condizione) verifica che sia falsa. Sono particolarmente adatti quando il risultato da verificare è già un booleano o quando si vuole esprimere una proprietà del sistema in forma logica diretta.

```
// Sintassi
assertTrue("Messaggio", condizione);   // fallisce se condizione == false
assertFalse("Messaggio", condizione);  // fallisce se condizione == true
```

Nella suite BnB questi metodi vengono impiegati in due contesti distinti:

- Verifica della disponibilità delle camere: assertTrue/assertFalse su bnb.verificaDisponibilita() prima e dopo una prenotazione, verificando che il sistema aggiorni correttamente lo stato di occupazione.
- Verifica della proprietà isOccupante(): in StatoPrenotazioneTest, assertTrue e assertFalse vengono applicati sistematicamente a tutti e otto gli stati, coprendo entrambe le classi di equivalenza (stati occupanti e stati liberatori).

```
// Esempi dalla suite BnB

// Verifica disponibilità camera (risultato booleano diretto)
assertTrue("Camera disponibile inizialmente",
bnb.verificaDisponibilita("Singola", inizio, fine)); assertFalse("Camera NON
disponibile dopo prenotazione",     bnb.verificaDisponibilita("Singola",
inizio, fine));

// Verifica isOccupante() per ogni stato
assertTrue("In ATTESA la camera è impegnata",  new StatoAttesa().isOccupante());
assertFalse("Dopo CHECK-OUT la camera è libera", new
StatoCheckOut().isOccupante());

// Verifica che dopo eliminazione la camera sia libera
assertFalse("Prenotazione eliminata non occupa la stanza", p.isOccupante());
```

3.5 assertNotNull — Verifica che un oggetto non sia null

assertNotNull(oggetto) verifica che il riferimento passato non sia null. È utile per garantire che un metodo abbia effettivamente creato e restituito un oggetto, prima di procedere a verificarne le proprietà. Il fallimento di assertNotNull indica generalmente che il metodo sotto test non ha completato la sua operazione principale.

```
// Sintassi
assertNotNull("Messaggio", oggetto); // fallisce se oggetto == null
```

Nella suite BnB viene utilizzato in BnBTest per verificare che il metodo richiediPrenotazione() abbia effettivamente creato e restituito un oggetto Prenotazione, e non null. Questo controllo è preliminare: se la prenotazione fosse null, qualsiasi successiva asserzione sulle sue proprietà lancerebbe una NullPointerException, rendendo il messaggio di errore poco chiaro.

```
// Esempio dalla suite BnB
Prenotazione p = bnb.richiediPrenotazione("Singola", inizio, fine, idCliente);

// Verifica preventiva: l'oggetto deve essere stato creato assertNotNull("La
prenotazione non dovrebbe essere null", p);

// Solo dopo questa verifica è sicuro accedere alle proprietà di p
assertFalse(bnb.verificaDisponibilita("Singola", inizio, fine));
```

3.6 fail — Forza il fallimento del test

fail(messaggio) è un metodo assert speciale che forza il fallimento immediato del test con il messaggio specificato. Non verifica una condizione: la sua sola esecuzione costituisce un fallimento. Viene tipicamente usato all'interno di blocchi catch per segnalare che si è verificata un'eccezione in un punto del test in cui non era attesa.

```
// Sintassi
fail("Messaggio che descrive perché il test è fallito");
```

Nella suite BnB viene impiegato in ServiziPagamentiTest per il test del caso positivo (testPagamentoValido). Il pattern utilizzato è il seguente: si esegue l'operazione in un blocco try, e se viene lanciata un'eccezione si invoca fail() per segnalare che un'operazione con dati validi non avrebbe dovuto produrre errori.

```
// Esempio dalla suite BnB — test del caso positivo del pagamento
@Test public void
testPagamentoValido() {
    ServiziPagamenti sp = new ServiziPagamenti(new HashMap<>());
try {
        // Con dati validi NON ci aspettiamo alcuna eccezione
sp.creaPagamento(555, "1234567812345878", 150.0f, null, null);
        // Se arriviamo qui, il test è passato (nessuna eccezione)
    } catch (Exception e) {
        //  Se  invece  viene  lanciata  un'eccezione,  il  test  deve  fallire
fail("Dati validi non devono lanciare eccezioni: " + e.getMessage());     }
}
```

3.7 @Test(expected) — Verifica che venga lanciata un'eccezione

Pur non essendo un metodo assert nel senso stretto, l'annotazione @Test(expected = TipoEccezione.class) costituisce il meccanismo JUnit 4 per verificare che un'operazione lanci un'eccezione di un tipo specifico. Il test ha successo se e solo se l'eccezione del tipo indicato viene lanciata durante l'esecuzione del metodo di test; fallisce sia se non viene lanciata alcuna eccezione, sia se viene lanciata un'eccezione di tipo diverso.

```
// Sintassi
@Test(expected = TipoEccezione.class) public
void nomeDelTest() {
    // Il codice qui deve lanciare TipoEccezione
    // Se non la lancia, il test FALLISCE
    // Se lancia un'eccezione diversa, il test FALLISCE
}
```

Nella suite BnB questa annotazione è utilizzata estensivamente per verificare che il sistema respinga correttamente input non validi e operazioni non permesse. È la modalità standard in JUnit 4 per i test negativi (testing delle classi non valide). Nella suite sono presenti 13 test annotati con @Test(expected = RuntimeException.class), distribuiti su tutti e quattro i package.

```
// Esempi dalla suite BnB

// 1. Dati di input non validi (classi di equivalenza non valide)
@Test(expected = RuntimeException.class) public
void testPrezzoNegativoNonValido() {
    new Camera("Economica", -10.0f, 1, 1); // prezzo < 0
}

// 2. Transizione di stato non consentita
@Test(expected = RuntimeException.class) public
void testPagamentoIllegaleInAttesa() {
    Prenotazione p = new Prenotazione(1,
        LocalDate.now().plusDays(1), LocalDate.now().plusDays(3), 1, 50.0f);
p.paga(); // illegale in stato ATTESA
}

// 3. Vincolo di business violato @Test(expected
= RuntimeException.class) public void
testPrenotazioneInSovrapposizione() {
    // Prima prenotazione: OK
bnb.richiediPrenotazione("Singola", inizio, fine, idCliente);
// Seconda prenotazione nelle stesse date: deve fallire
bnb.richiediPrenotazione("Singola", inizio, fine, idCliente);
}
```

3.8 Riepilogo dei metodi assert utilizzati nella suite

Metodo Assert	Scopo	Utilizzi nella suite
assertEquals(exp, act)	Verifica uguaglianza tra valori	Stato prenotazione, attributi Camera, date
assertEquals(exp, act, delta)	Uguaglianza float con tolleranza	Calcolo prezzoTotale (delta = 0.001)
assertTrue(condizione)	Verifica che la condizione sia vera	Disponibilità camera, isOccupante(), autenticazione
assertFalse(condizione)	Verifica che la condizione sia falsa	Camera occupata, isOccupante() stati terminali
sertNotNull(oggetto)	Verifica che l'oggetto non sia null	Risultato di richiediPrenotazione()
fail(messaggio)	Forza il fallimento del test	Blocco catch di testPagamentoValido()
@Test(expected=...)	Verifica che venga lanciata un'eccezione	13 test negativi su input e transizioni illegali

---

4. Architettura del Sistema e Pattern Adottati

Prima di illustrare la suite di test, è opportuno descrivere i pattern architetturali impiegati, poiché le scelte progettuali dei test riflettono direttamente la struttura del codice di produzione.

4.1 Pattern Singleton — Classe BnB

La classe BnB adotta il pattern Singleton, garantendo l'esistenza di una sola istanza durante l'esecuzione. Questo è giustificato dalla natura del dominio in un sistema centralizzato gestisce tutte le camere, i clienti e le prenotazioni.

Il Singleton introduce però una criticità nel testing: l'istanza condivisa tra i test può conservare stato residuo, compromettendo l'isolamento. La soluzione adottata è l'uso del metodo @After, che azzera il campo privato istanza dopo ogni singolo test:

```
@After
public void tearDown() throws Exception {
    Field instance = BnB.class.getDeclaredField("istanza");
instance.setAccessible(true);
```

```
        instance.set(null, null);  // Reset del Singleton
    }
```

Questa tecnica garantisce che ogni test parta da un sistema pulito senza modificare il codice di produzione. L'approccio evita dipendenze tra test consecutivi e garantisce il principio di test isolation, fondamentale nei test unitari: ogni test deve essere indipendente dagli altri e produrre lo stesso risultato indipendentemente dall'ordine di esecuzione.

4.2 Pattern Facade — Classe BnB come punto di accesso

La classe BnB funge da Facade, nascondendo la complessità dei sottosistemi interni (ServiziClienti, ServiziPrenotazioni, ServiziPagamenti). I test di integrazione del package beb.facade interagiscono con il sistema tramite questa unica interfaccia, verificando il comportamento complessivo senza dipendere dall'implementazione dei singoli servizi.

4.3 Pattern State — Gestione degli stati della prenotazione

Il ciclo di vita di una prenotazione è modellato attraverso il pattern State. Ogni stato implementa un'interfaccia comune che definisce le operazioni ammissibili. Quando un'operazione non è consentita nello stato corrente (es. check-out prima del check-in), lo stato lancia una RuntimeException, intercettata nei test tramite @Test(expected = RuntimeException.class). Il pattern State facilita il testing perché ogni stato può essere verificato in modo indipendente, consentendo di testare separatamente le transizioni di stato e le operazioni consentite.

Stato	isOccupante()	Operazioni ammesse	Tipo
ATTESA	true	approva(), elimina()	Transitorio
APPROVATA	true	paga(), elimina()	Transitorio
PRENOTATA	true	checkIn(), modifica(), cancella()	Transitorio
MODIFICATA	true	esitoModifica()	Transitorio
CHECK-IN	true	checkOut()	Transitorio
CHECK-OUT	false	Nessuna	Terminale
CANCELLATA	false	Nessuna	Terminale
ELIMINATA	false	Nessuna	Terminale

5. Struttura della Suite di Test

La suite è organizzata in quattro package che corrispondono ai livelli di astrazione del sistema, secondo una struttura a piramide: alla base si trovano i test unitari, più numerosi e veloci, mentre al vertice si trovano i test di integrazione, più completi ma più costosi da eseguire.

```
Test Suite BnB
 |
 +-- beb.dominio      [Unitari]   CameraTest, PrenotazioneTest
|
 +-- beb.stati        [Unitari]   StatoPrenotazioneTest
|
 +-- beb.servizi      [Unitari]   ServiziClientiTest,
 |                               ServiziPrenotazioniTest,
 |                               ServiziPagamentiTest
|
 +-- beb.facade       [Integr.]   BnBTest, ProcessoSoggiornoTest,
                                  CancellazioneTest, CheckOutTest,
                                  ModificaPrenotazioneTest,
                                  ValidazioneAvanzataTest
```

Classe di Test	Package	Tipo	N° Test
CameraTest	beb.dominio	Unitario	2
PrenotazioneTest	beb.dominio	Unitario	3
StatoPrenotazioneTest	beb.stati	Unitario	4
ServiziClientiTest	beb.servizi	Unitario	2
ServiziPrenotazioniTest	beb.servizi	Unitario	1 (4 casi)
ServiziPagamentiTest	beb.servizi	Unitario	4
BnBTest	beb.facade	Integrazione	2
ProcessoSoggiornoTest	beb.facade	Integrazione	2
CancellazioneTest	beb.facade	Integrazione	2
CheckOutTest	beb.facade	Integrazione	2

ModificaPrenotazioneTest	beb.facade	Integrazione	2
ValidazioneAvanzataTest	beb.facade	Integrazione	2

I test dei package beb.dominio, beb.stati e beb.servizi sono test unitari che verificano componenti in isolamento; i test del package beb.facade sono test di integrazione che verificano i flussi completi.

6. Descrizione Dettagliata dei Test

6.1 Package beb.dominio

I test di questo package verificano le entità fondamentali del dominio. Essendo test unitari puri, non richiedono né Singleton né @After, ogni test istanzia i propri oggetti in isolamento completo.

6.1.1 CameraTest

La classe Camera rappresenta una tipologia di alloggio con quattro attributi: tipo (stringa), prezzo per notte (float), capienza (int) e quantità disponibile (int). Il testing adotta la partizione in classi di equivalenza per il parametro prezzo, identificando una classe valida CV1 = {prezzoPerNotte > 0} e una classe non valida CNV1 = {prezzoPerNotte ≤ 0}.

```
// CV1: dati tutti validi — verifica costruttore completo
@Test
public void testDatiCameraCorretti() {
    Camera c = new Camera("Suite Royal", 250.0f, 4, 2);
assertEquals("Suite Royal", c.getTipoCamera());     assertEquals(250.0f,
c.getPrezzoPerNotte(), 0.001);     assertEquals("La capienza dovrebbe
essere 4", 4, c.getCapienza());
    assertEquals("La quantità disponibile dovrebbe essere 2", 2, c.getQuantita());
}

// CNV1: prezzo negativo — boundary value al di sotto del minimo valido
@Test(expected = RuntimeException.class) public
void testPrezzoNegativoNonValido() {
    new Camera("Economica", -10.0f, 1, 1);
}
```

Il secondo test applica l'analisi dei valori limite: il valore -10 è un rappresentante della classe non valida CNV1 e verifica che il costruttore lanci una RuntimeException prima ancora che l'oggetto venga creato. In una versione precedente del test mancavano le asserzioni su capienza e quantità: la lacuna è stata corretta per garantire la copertura completa di tutti i parametri del costruttore.

6.1.2 PrenotazioneTest

La classe PrenotazioneTest si concentra sulla logica di calcolo del prezzo totale e sulla validazione delle date. Le classi di equivalenza per la durata del soggiorno sono: CV1 = {numNotti = 1} (caso limite inferiore), CV2 = {numNotti > 1} (caso standard), CNV1 = {dataFine < dataInizio} (date invertite).

```
// CV2: soggiorno di 3 notti — caso standard
@Test
public void testCalcoloPrezzoSoggiorno() {
LocalDate inizio = LocalDate.of(2025, 6, 10);
    LocalDate fine   = LocalDate.of(2025, 6, 13);
    Prenotazione p = new Prenotazione(1, inizio, fine, 1, 80.0f);     // 3
notti * 80.0 = 240.0 — delta per imprecisioni float
assertEquals("Prezzo totale non corretto", 240.0f, p.getPrezzoTotale(),
0.001); }

// CV1: boundary value — soggiorno minimo (1 notte)
@Test
public void testSoggiornoUnaSolaNotte() {
LocalDate oggi   = LocalDate.now();
    LocalDate domani = oggi.plusDays(1);
    Prenotazione p = new Prenotazione(2, oggi, domani, 1, 100.0f);
assertEquals("Una notte deve coincidere col prezzo base",
            100.0f, p.getPrezzoTotale(), 0.001);
}

// CNV1: date invertite — il costruttore deve lanciare eccezione
@Test(expected = RuntimeException.class) public
void testDateInvertite() {
    LocalDate inizio = LocalDate.now().plusDays(10);
    LocalDate fine   = LocalDate.now().plusDays(5); // fine < inizio
new Prenotazione(1, inizio, fine, 1, 100.0f);
}
```

L'utilizzo del parametro delta (0.001) nell'assertEquals è motivato dalla natura del tipo float, soggetto a errori di rappresentazione in virgola mobile. Come riportato nel commento nel codice, una futura evoluzione dovrebbe adottare BigDecimal per eliminare questa necessità. Il test testDateInvertite copre il caso limite più pericoloso: senza validazione nel costruttore, il calcolo della durata produrrebbe un valore negativo, con conseguente prezzo totale erroneo.

6.2 Package beb.stati

Questo package contiene test unitari che verificano il comportamento degli oggetti di stato in completo isolamento, senza passare dalla Facade.

6.2.1 StatoPrenotazioneTest

I test coprono le transizioni lecite, quelle illegali e le proprietà invarianti degli stati. Applicando la tecnica del testing basato sugli scenari, ogni transizione del diagramma di stato viene esercitata almeno una volta.

```
// Transizione ATTESA → APPROVATA: caso positivo
@Test
public void testInizializzazioneEApprovazione() {
    Prenotazione p = new Prenotazione(1,
        LocalDate.now().plusDays(1), LocalDate.now().plusDays(3), 1, 50.0f);
assertEquals("ATTESA", p.getStatoNome());     p.approva();
    assertEquals("APPROVATA", p.getStatoNome());
}

// Transizione ATTESA → paga(): operazione illegale
@Test(expected = RuntimeException.class) public
void testPagamentoIllegaleInAttesa() {
    Prenotazione p = new Prenotazione(1,
        LocalDate.now().plusDays(1), LocalDate.now().plusDays(3), 1, 50.0f);
p.paga(); // deve lanciare RuntimeException
}
```

Il test seguente verifica la proprietà isOccupante() per tutti e otto gli stati, garantendo che la logica di disponibilità delle camere sia corretta. Si tratta di un test di copertura minima delle classi di equivalenza: gli stati sono partizionati in due classi (occupanti / non occupanti) e per ciascuna viene verificato almeno un rappresentante.

```
// Verifica sistematica di isOccupante() su tutti gli stati
@Test
public void testStatoOccupaCamera() {
    // Classe CV-occupanti: devono restituire TRUE
assertTrue(new StatoAttesa().isOccupante());     assertTrue(new
StatoPrenotata().isOccupante());     assertTrue(new
StatoCheckIn().isOccupante());

    // Classe CV-liberi: devono restituire FALSE
assertFalse(new StatoCancellata().isOccupante());
assertFalse(new StatoEliminata().isOccupante());
assertFalse(new StatoCheckOut().isOccupante());
}
// Rollback delle date in caso di modifica rifiutata
@Test
public void testRipristinoDateInStatoModificata() {
    LocalDate dataOriginale = LocalDate.of(2025, 6, 1);
Prenotazione p = new Prenotazione(1,         dataOriginale,
dataOriginale.plusDays(2), 1, 100.0f);
p.setDataInizioPrecedente(dataOriginale);
    p.setDataInizio(LocalDate.of(2025, 12, 25));
StatoModificata sm = new StatoModificata();
sm.esitoModifica(p, false); // rifiuto → rollback
    assertEquals("Data ripristinata", dataOriginale, p.getDataInizio());
}
```

6.3 Package beb.servizi

I test di questo package verificano i servizi applicativi in isolamento, iniettando le dipendenze tramite i costruttori. Questo approccio evita l'utilizzo del Singleton ed elimina qualsiasi effetto collaterale tra i test.

6.3.1 ServiziClientiTest

Le classi di equivalenza per l'operazione di ricerca cliente sono: CV1 = {cliente registrato nel sistema}, CNV1 = {cliente non registrato}. Per l'autenticazione: CV1 = {password corretta}, CNV1 = {password errata}, CNV2 = {utente inesistente}.

```
// CNV1: ricerca cliente inesistente — mappa vuota
@Test(expected = RuntimeException.class) public
void testRicercaClienteInesistente() {
    ServiziClienti sc = new ServiziClienti(new HashMap<>());
sc.cercaCliente("fantasma@test.it"); // deve lanciare eccezione
}

// CV1 + CNV1: autenticazione corretta e con password errata
@Test
public void testAutenticazioneCorretta() {
    ServiziClienti sc = new ServiziClienti(new HashMap<>());
sc.registraCliente("Test", "test@email.it", "1234");
assertTrue(sc.autenticaCliente("test@email.it", "1234"));    // CV1
assertFalse(sc.autenticaCliente("test@email.it", "sbagliata")); // CNV1
}
```

6.3.2 ServiziPrenotazioniTest

Questo test verifica la logica di rilevamento delle sovrapposizioni di date, un requisito critico per l'integrità del sistema. Il dominio degli input è partizionato in quattro classi, ciascuna rappresentativa di un tipo distinto di relazione temporale tra due intervalli. Sono stati scelti rappresentanti ai confini degli intervalli (boundary values), in quanto la logica di confronto è particolarmente soggetta a errori sui valori esatti di inizio e fine.

```
// Prenotazione esistente: dal 10 al 15 giugno sp.richiediPrenotazioni(1,
LocalDate.of(2025,6,10), LocalDate.of(2025,6,15), 1,
50.0f);
// CASO A: [5-10] — fine coincide con inizio dell'esistente → NO sovrapposizione
// Boundary: il confine esatto non è sovrapposizione (semantica alberghiera) int
cA = sp.contaPrenotazioni(1, of(2025,6,5), of(2025,6,10)); assertEquals(0, cA);

// CASO B: [15-20] — inizio coincide con fine dell'esistente → NO sovrapposizione
int cB = sp.contaPrenotazioni(1, of(2025,6,15), of(2025,6,20)); assertEquals(0,
cB);

// CASO C: [14-18] — parzialmente sovrapposto → sovrapposizione RILEVATA int
cC = sp.contaPrenotazioni(1, of(2025,6,14), of(2025,6,18)); assertEquals(1,
cC);
```

```
// CASO D: [11-14] — totalmente incluso → sovrapposizione RILEVATA int
cD = sp.contaPrenotazioni(1, of(2025,6,11), of(2025,6,14));
assertEquals(1, cD);
```

La scelta progettuale documentata dai Casi A e B è fondamentale: due prenotazioni i cui intervalli si toccano esattamente al confine non sono considerate sovrapposte. Questo riflette la semantica reale del dominio alberghiero, dove il check-out mattutino libera la stanza per un check-in dello stesso giorno. I Casi C e D coprono invece le due classi di sovrapposizione effettiva (parziale e totale interna).

6.3.3 ServiziPagamentiTest

La validazione del pagamento coinvolge due input con vincoli precisi: il numero di carta (stringa di esattamente 16 cifre) e il CVV (intero nell'intervallo [100, 999]). L'analisi dei valori limite produce le seguenti classi per il CVV:

Classe	Valori	Tipo	Test
CNV-basso	CVV < 100 (es. 99)	Non valida	testCvvNonValidoBasso
CV-valido	100 ≤ CVV ≤ 999 (es. 555)	Valida	testPagamentoValido
CNV-alto	CVV > 999 (es. 1000)	Non valida	testCvvNonValidoAlto
Classe	**Valori**	**Tipo**	**Test**
CNV-carta	len(carta) ≠ 16	Non valida	testCartaTroppoCorta

```
// CNV-carta: numero di carta da 15 cifre (boundary: 16-1)
@Test(expected = RuntimeException.class)
public void testCartaTroppoCorta() {
new ServiziPagamenti(new HashMap<>())
        .creaPagamento(123, "123456789012345", 100.0f, null, null);
}

// CNV-basso: CVV = 99 (boundary: min-1 = 100-1)
@Test(expected = RuntimeException.class) public
void testCvvNonValidoBasso() {     new
ServiziPagamenti(new HashMap<>())
        .creaPagamento(99, "1234567812345678", 100.0f, null, null);
}

// CNV-alto: CVV = 1000 (boundary: max+1 = 999+1)
@Test(expected = RuntimeException.class) public
void testCvvNonValidoAlto() {     new
ServiziPagamenti(new HashMap<>())
        .creaPagamento(1000, "1234567812345678", 100.0f, null, null);
}

// CV-valido: caso positivo — nessuna eccezione attesa
@Test
```

```
public void testPagamentoValido() {     try {
new ServiziPagamenti(new HashMap<>())
            .creaPagamento(555, "1234567812345878", 150.0f, null, null);
    } catch (Exception e) {         fail("Dati validi non devono lanciare
eccezioni: " + e.getMessage());     }
}
```

Si noti che la rappresentazione del CVV come int esclude i codici con zero iniziale (es. 099), poiché il valore numerico 99 sarebbe fuori dal range valido. Una futura evoluzione dovrebbe adottare il tipo String con validazione tramite espressione regolare \d{3}, che consentirebbe di gestire correttamente tutti i CVV a tre cifre inclusi quelli con zero iniziale.

6.4 Package beb.facade

I test di integrazione di questo package verificano i flussi applicativi completi attraverso la Facade BnB. Ogni classe è dotata di @Before per il setup e di @After per il reset del Singleton tramite reflection. L'approccio adottato è il testing basato sugli scenari dei casi d'uso: ogni flusso di business rilevante viene modellato come sequenza di chiamate alla Facade con verifica dello stato dopo ogni passo.

6.4.1 BnBTest

Verifica la disponibilità delle camere e il blocco delle prenotazioni sovrapposte, che costituiscono le funzionalità di base della Facade.

```
// Scenario: disponibilità → prenotazione → indisponibilità
@Test
public void testPrenotazioneEVerificaDisponibilita() {
int idCliente = bnb.getIdCliente("mario@email.it");
LocalDate inizio = LocalDate.now().plusDays(10);
    LocalDate fine   = LocalDate.now().plusDays(15);

    // 1. La camera deve essere inizialmente disponibile
assertTrue(bnb.verificaDisponibilita("Singola", inizio, fine));

    // 2. Dopo la prenotazione (StatoAttesa.isOccupante()=true)
    Prenotazione p = bnb.richiediPrenotazione("Singola", inizio, fine, idCliente);
assertNotNull(p);

    // 3. La camera non è più disponibile
    assertFalse(bnb.verificaDisponibilita("Singola", inizio, fine));
}

// Scenario negativo: seconda prenotazione sulle stesse date deve fallire
@Test(expected = RuntimeException.class) public
void testPrenotazioneInSovrapposizione() {
int id = bnb.getIdCliente("mario@email.it");
```

```
    LocalDate in = LocalDate.now().plusDays(20), fi =
LocalDate.now().plusDays(25);
    bnb.richiediPrenotazione("Singola", in, fi, id); // OK
    bnb.richiediPrenotazione("Singola", in, fi, id); // deve fallire }
```

6.4.2 ProcessoSoggiornoTest

Testa il ciclo di vita completo di una prenotazione (scenario principale) e la transizione illegale da APPROVATA a CHECK-IN senza pagamento (scenario negativo).

```
// Scenario principale: ATTESA→APPROVATA→PRENOTATA→CHECK-IN
@Test
public void testCicloDiVitaCompleto() {
    int idC = bnb.getIdCliente("luigi@email.it");
    LocalDate oggi = LocalDate.now();
    Prenotazione p = bnb.richiediPrenotazione("Suite",oggi,oggi.plusDays(2),idC);
assertEquals("ATTESA",     p.getStatoNome());
bnb.aggiornaRichiesta(p.getIdPrenotazione(), "APPROVATA");
assertEquals("APPROVATA",  p.getStatoNome());
bnb.pagaRichiesta(p.getIdPrenotazione(), "1234567812345678", 123);
assertEquals("PRENOTATA",  p.getStatoNome());
bnb.confermaCheckIn(p.getIdPrenotazione());    assertEquals("CHECK-IN",
p.getStatoNome());
}

// Scenario negativo: check-in senza pagamento
@Test(expected = RuntimeException.class) public void
testCheckInImpossibileSenzaPagamento() {     int idC =
bnb.getIdCliente("luigi@email.it");     Prenotazione p
= bnb.richiediPrenotazione("Suite",
LocalDate.now(), LocalDate.now().plusDays(1), idC);
bnb.aggiornaRichiesta(p.getIdPrenotazione(),
"APPROVATA");
bnb.confermaCheckIn(p.getIdPrenotazione()); //
eccezione attesa }
```

6.4.3 CancellazioneTest

Verifica le diverse politiche di cancellazione in base al ruolo (cliente vs gestore) e allo stato della prenotazione.

```
// Il cliente NON può cancellare dopo il check-in @Test(expected
= RuntimeException.class)
public void testClienteNonPuoCancellareDopoCheckIn() {
int idC = bnb.getIdCliente("marco@email.it");
    Prenotazione p = bnb.richiediPrenotazione("Matrimoniale",
LocalDate.now(), LocalDate.now().plusDays(1), idC);
bnb.aggiornaRichiesta(p.getIdPrenotazione(), "APPROVATA");
bnb.pagaRichiesta(p.getIdPrenotazione(), "1234123412341234", 999);
bnb.confermaCheckIn(p.getIdPrenotazione());
    bnb.cancellaPrenotazioneCliente(p.getIdPrenotazione()); // eccezione
}
```

```
// Il GESTORE può sempre eliminare, in qualsiasi stato
@Test
public void testGestorePuoSempreEliminare() {
int idC = bnb.getIdCliente("marco@email.it");
    Prenotazione p = bnb.richiediPrenotazione("Matrimoniale",
LocalDate.now(), LocalDate.now().plusDays(1), idC);
bnb.cancellaPrenotazioneGestore(p.getIdPrenotazione());
assertEquals("ELIMINATA", p.getStatoNome());     assertFalse(p.isOccupante());
// camera liberata

}
```

6.4.4 CheckOutTest

Verifica il flusso corretto di check-out e il divieto di effettuarlo senza aver prima eseguito il checkin.

```
// Flusso corretto: CHECK-IN → CHECK-OUT
@Test
public void testFlussoCheckOutCorretto() {
int idC = bnb.getIdCliente("sara@email.it");
    LocalDate oggi = LocalDate.now();
Prenotazione p =
bnb.richiediPrenotazione("Singola",oggi,oggi.plusDays(1),idC);
bnb.aggiornaRichiesta(p.getIdPrenotazione(), "APPROVATA");
bnb.pagaRichiesta(p.getIdPrenotazione(), "1111222233334444", 123);
bnb.confermaCheckIn(p.getIdPrenotazione());     assertEquals("CHECK-IN",
p.getStatoNome());     bnb.confermaCheckOut(p.getIdPrenotazione());
assertEquals("CHECK-OUT", p.getStatoNome());
assertFalse(p.isOccupante()); // camera liberata
}

// Check-out senza check-in: operazione illegale
@Test(expected = RuntimeException.class) public void
testImpossibileCheckOutSenzaCheckIn() {     int idC =
bnb.getIdCliente("sara@email.it");     Prenotazione p =
bnb.richiediPrenotazione("Singola",         LocalDate.now(),
LocalDate.now().plusDays(1), idC);
bnb.aggiornaRichiesta(p.getIdPrenotazione(), "APPROVATA");
bnb.pagaRichiesta(p.getIdPrenotazione(), "1111222233334444", 123);
bnb.confermaCheckOut(p.getIdPrenotazione()); // eccezione attesa
}
```

6.4.5 ModificaPrenotazioneTest

Verifica il meccanismo di modifica delle date, includendo il rollback in caso di rifiuto da parte del gestore e il divieto di modifica in stati non consentiti.

```
// Modifica rifiutata: le date originali devono essere ripristinate
@Test
public void testModificaRifiutataRipristinoDate() {
int idC = bnb.getIdCliente("anna@email.it");
LocalDate inizioOrig = LocalDate.now().plusDays(10);
    LocalDate fineOrig   = LocalDate.now().plusDays(12);
```

```
    Prenotazione p = bnb.richiediPrenotazione("Doppia",inizioOrig,fineOrig,idC);
bnb.aggiornaRichiesta(p.getIdPrenotazione(), "APPROVATA");
bnb.pagaRichiesta(p.getIdPrenotazione(), "1111222233334444", 555);
    // Richiesta di modifica verso nuove date
    bnb.richiestaModificaPrenotazione(p.getIdPrenotazione(),
LocalDate.now().plusDays(20), LocalDate.now().plusDays(22));
assertEquals("MODIFICATA", p.getStatoNome());     // Rifiuto da parte
del gestore → rollback
bnb.aggiornaRichiestaModificata(p.getIdPrenotazione(), false);
assertEquals("PRENOTATA",  p.getStatoNome());
assertEquals(inizioOrig, p.getDataInizio()); // data ripristinata
assertEquals(fineOrig,   p.getDataFine());
}

// Modifica in stato ATTESA: non consentita
@Test(expected = RuntimeException.class) public
void testModificaNonConsentitaInAttesa() {
int idC = bnb.getIdCliente("anna@email.it");
    Prenotazione p = bnb.richiediPrenotazione("Doppia",
        LocalDate.now().plusDays(5), LocalDate.now().plusDays(7), idC);
bnb.richiestaModificaPrenotazione(p.getIdPrenotazione(),
LocalDate.now().plusDays(10), LocalDate.now().plusDays(12));
}
```

6.4.6 ValidazioneAvanzataTest

Verifica due validazioni di dominio avanzate: il rifiuto di date nel passato e l'impossibilità di modificare una prenotazione verso date già occupate da un'altra prenotazione attiva.

```
// Date nel passato: non accettabili dal sistema
@Test(expected = RuntimeException.class) public void
testPrenotazioneNelPassato() {     int idC =
bnb.getIdCliente("test@test.it");     LocalDate
passato = LocalDate.now().minusYears(1);
    bnb.richiediPrenotazione("Singola", passato, passato.plusDays(5), idC);
}

// Modifica su date già occupate da altra prenotazione @Test(expected
= RuntimeException.class)
public void testModificaSuDateGiaOccupate() {
int idC = bnb.getIdCliente("test@test.it");
    // p1 occupa 10-15
    LocalDate in1=LocalDate.now().plusDays(10), fi1=LocalDate.now().plusDays(15);
Prenotazione p1 = bnb.richiediPrenotazione("Singola",in1,fi1,idC);
bnb.aggiornaRichiesta(p1.getIdPrenotazione(), "APPROVATA");
bnb.pagaRichiesta(p1.getIdPrenotazione(), "1111222233334444", 123);
    // p2 occupa 20-25
    LocalDate in2=LocalDate.now().plusDays(20), fi2=LocalDate.now().plusDays(25);
    Prenotazione p2 = bnb.richiediPrenotazione("Singola",in2,fi2,idC);
// Tentativo di spostare p2 su 10-15 (già occupate da p1) → eccezione
bnb.richiestaModificaPrenotazione(p2.getIdPrenotazione(), in1, fi1);
}
```

In una versione precedente di questo test veniva utilizzato un numero di carta non valido (4 cifre) per il pagamento di p1. Il test avrebbe potuto fallire per la ragione sbagliata, ovvero per il rifiuto del pagamento anziché per la sovrapposizione delle date. Il difetto è stato corretto utilizzando una carta

valida da 16 cifre, garantendo che l'eccezione sia lanciata esclusivamente dalla logica di controllo della disponibilità.

---

7. Scelte Progettuali della Suite di Test

7.1 Classificazione dei test rispetto alle tecniche di testing

La tabella seguente classifica i test della suite in base alla tecnica adottata. La colonna 'Tecnica' riporta il nome formale della tecnica; si noti che StatoPrenotazioneTest adotta la tecnica Test sulle transizioni di stato formalmente distinta dalla semplice partizione in classi di equivalenza.

Test / Classe	Tecnica Black Box	Classi di equivalenza
ServiziPagamentiTest	Boundary Value Analysis	CVV: {<100}, {100-999}, {>999}; Carta: {≠16 cifre}, {16 cifre}
ServiziPrenotazioniTest	Boundary Value Analysis	Date: {tocca-confine}, {sovrappostoparziale}, {incluso}
PrenotazioneTest	Classi di equivalenza + BVA	Notti: {1}, {>1}, {<0}; Float: delta per rappresentazione
CameraTest	Classi di equivalenza	Prezzo: {>0} valida, {≤0} non valida
StatoPrenotazioneTest	State Transition Testing	8 stati con proprietà isOccupante e transizioni
BnBTest, ProcessoSoggiornoTest, ecc.	Testing basato sugli scenari	Scenari principali e alternativi del ciclo di vita

7.2 Gestione del tipo float

Il tipo float è soggetto a errori di rappresentazione in virgola mobile. Per questa ragione tutte le asserzioni su valori monetari utilizzano assertEquals(expected, actual, delta) con delta = 0.001. Il codice documenta esplicitamente questa scelta e suggerisce l'adozione di BigDecimal in future iterazioni, che garantirebbe precisione arbitraria eliminando la necessità del delta.

7.3 Isolamento tramite reset del Singleton

L'utilizzo del Singleton rende i test di integrazione potenzialmente dipendenti dall'ordine di esecuzione. La soluzione adottata resettare il campo privato istanza tramite il metodo @After garantisce l'isolamento senza modificare il codice di produzione. Questa tecnica è una pratica

consolidata nel testing di sistemi con stato condiviso, ed è preferibile a soluzioni alternative come l'aggiunta di un metodo reset() pubblico nel codice di produzione, che ne altererebbe il design.

7.4 Separazione tra test unitari e test di integrazione

La suite distingue nettamente i due livelli. I test unitari (beb.dominio, beb.stati, beb.servizi) iniettano le dipendenze manualmente tramite i costruttori, senza accedere al Singleton. I test di integrazione (beb.facade) esercitano il sistema nel suo complesso attraverso la Facade. Un difetto rilevato solo nei test di integrazione, e non in quelli unitari, è probabilmente legato all'interazione tra i componenti piuttosto che alla logica interna di ciascuno.

7.5 Copertura dei casi limite

Una particolare attenzione è stata dedicata ai boundary values, che storicamente concentrano la maggior parte dei difetti:

- CVV ai limiti esatti del range: 99 (min-1), 100 (min), 999 (max), 1000 (max+1).
- Numero di carta: 15 cifre (min-1 rispetto alla lunghezza valida di 16).
- Date di prenotazione ai confini dell'intervallo: confine coincidente (non sovrapposizione), sovrapposizione di un solo giorno.
- Soggiorno di una sola notte (durata minima).
- Date nel passato (violazione del vincolo temporale minimo).
- Date invertite nel costruttore di Prenotazione.

8. Conclusioni

In conclusione, la campagna di collaudo progettata e realizzata per il sistema di gestione del Bed & Breakfast copre in modo esaustivo i principali scenari operativi. Il lavoro svolto ha permesso di verificare con successo sia il comportamento isolato dei singoli moduli, sia la correttezza dei flussi applicativi completi, simulando l'intero ciclo di vita di una prenotazione: dalla richiesta iniziale, alla gestione delle date, fino al saldo e al rilascio della camera. Inoltre, la netta separazione architettonica tra le verifiche di unità e le verifiche di integrazione ha favorito una rapida individuazione e localizzazione degli eventuali difetti.

In particolare, l'applicazione rigorosa della partizione in classi di equivalenza e l'analisi dei valori limite hanno garantito una validazione profonda degli inserimenti da parte dell'utente, concentrando i controlli proprio sui confini degli intervalli numerici e temporali, dove si annidano più frequentemente gli errori. A questo si aggiunge l'analisi basata sulle transizioni di stato, che ha assicurato il rispetto rigoroso della sequenza delle operazioni consentite, impedendo al sistema di compiere azioni non valide in momenti inopportuni.

Infine, ha assunto un ruolo centrale l'approccio ibrido (Grey-Box) adottato per la stesura del codice di verifica. La progettazione dei casi di prova ha seguito una logica a Black-Box, basandosi esclusivamente sui requisiti esterni dell'interfaccia. Tuttavia, durante la fase di programmazione pratica, si è ricorsi a tecniche di ispezione del codice (White-Box) per risolvere criticità complesse. Nello specifico, l'utilizzo dei meccanismi di riflessione del linguaggio per azzerare l'istanza unica del sistema tra un'esecuzione e l'altra ha garantito la totale indipendenza e l'isolamento dei controlli. Questo ha permesso di preservare l'integrità del codice originale, dimostrando come una solida strategia di validazione possa adattarsi alla struttura dell'applicazione senza forzarne o comprometterne le scelte architetturali di base.